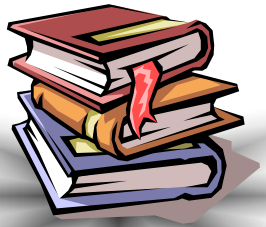
FPT Education

FPT UNIVERSITY

LOS201
ASSIGNMENT 2
Write Embedded Application
Sensor Monitoring System



Lecturer: Mr. Phạm Thế Vinh

Group 3:

No.	Name	Student ID
1	Nguyễn Duy Khoa	SE200494
2	Phạm Võ Hoàng Nhân	SE200778
3	Nguyễn Cao Kỳ	SE201611
4	Ngô Từ Nhã Quyên	SE201483

Monday, October 20, 2026

Assignment 02 Directory Structure

The project is organized by function: kernel driver source, multithreaded application, ARM root filesystem, build outputs, and test logs. The structure below summarizes filenames used during implementation. Some logs may remain in QEMU until transferred to the Ubuntu host.

```
assignment02/
├── app/
│   ├── sms.h
│   ├── sms_app.c
│   ├── sensor_thread.c
│   ├── processor_thread.c
│   ├── alert_thread.c
│   ├── logger_thread.c
│   ├── circular_buffer.c
│   ├── Makefile
│   ├── sms_app                 (ARM)
│   ├── sms_app_x86             (x86_64)
│   └── sms_app_debug           (x86_64, GDB)
├── driver/
│   ├── sms_sensor.h
│   ├── sms_sensor_driver.c
│   ├── Makefile
│   └── sms_sensor.ko
├── kernel/
│   └── linux-5.15/             (dedicated Assignment 02 kernel)
├── rootfs/                    (ARM root filesystem)
│   ├── dev/
│   ├── lib/
│   ├── proc/
│   ├── sys/
│   ├── usr/bin/sms_app
│   └── init
├── output/
│   └── initramfs_asgn2.cpio.gz
├── logs/
│   ├── valgrind_sms.txt
│   ├── gdb_sms.txt
│   ├── strace_sms.txt
│   ├── perf_sms.txt
│   ├── perf_sms.data
│   └── jitter_report.txt       (if transferred to host)
└── Report/                   (exported report PDF)
```

Directory and File Functions

Directory or file	Purpose
app/sms_app.c and sms.h	Main function, argument handling, data structures, and shared declarations.
app/*_thread.c	Four threads for sampling, processing, receiving alerts, and logging.
app/circular_buffer.c	Shared circular buffer and thread synchronization.
app/sms_app, sms_app_x86, sms_app_debug	ARM build for QEMU, x86 build for host testing, and debug build for GDB.
driver/sms_sensor_driver.c, sms_sensor.h, sms_sensor.ko	Sensor driver source, ioctl header, and kernel module.
kernel/linux-5.15/	Dedicated kernel build with CONFIG_POSIX_MQUEUE enabled.
rootfs/ and output/	ARM root filesystem and initramfs image for booting QEMU.
logs/	Valgrind, GDB, strace, perf, and jitter evidence.
demo/ and bao_cao/	Submission locations for the demo video and report PDF.

1. System Overview

Assignment 02 implements a simulated sensor monitoring system using the sms_sensor.ko kernel driver and the sms_app userspace application. Temperature and humidity are sampled periodically, processed by multiple threads, checked against thresholds, and logged. The system is cross-compiled for ARMv7 and tested in QEMU.

1.1. System Architecture

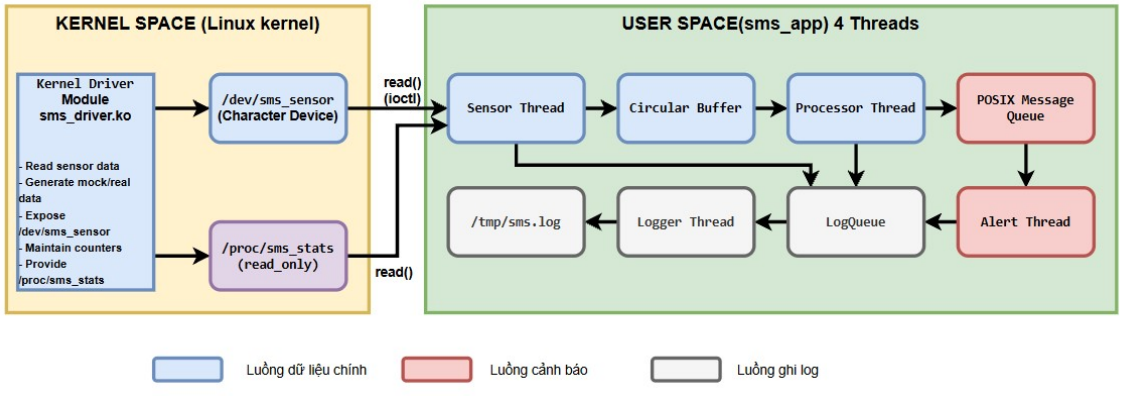


Figure 1. Overall System Architecture with Kernel Space and User Space

The driver exposes /dev/sms_sensor for reading samples and /proc/sms_stats for viewing statistics. The Sensor Thread sends samples to the Circular Buffer. The Processor Thread averages 10 samples and detects TEMP_HIGH, HUMID_HIGH, and TEMP_SPIKE events. The Alert Thread receives alerts through a POSIX Message Queue. The Logger Thread writes messages from LogQueue to /tmp/sms.log.

1.2. Application State Diagram

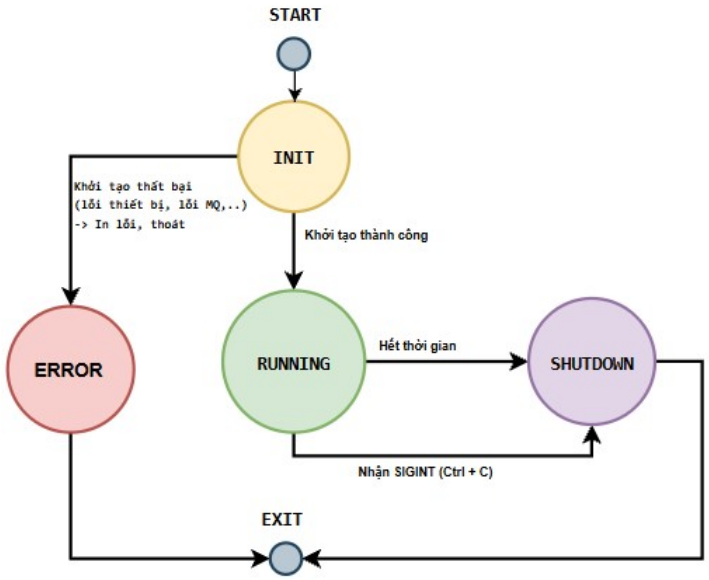


Figure 2. Application States INIT RUNNING SHUTDOWN ERROR and EXIT

INIT creates queues, mutexes, and threads. Successful initialization transitions the application to RUNNING. A timeout or SIGINT/SIGTERM triggers SHUTDOWN, which joins threads, flushes logs, and releases resources. Initialization failures trigger cleanup before exit.

2. Task 1 Kernel Driver and Multithreaded Application

2.1. Implementation

The sms_sensor.ko kernel module registers the /dev/sms_sensor misc device, supports read() and ioctl(SMS_SET_INTERVAL), and exposes statistics through /proc/sms_stats. The application has four worker threads: Sensor, Processor, Alert, and Logger. A mutex and condition variable synchronize access to the Circular Buffer.

During the ARM build, assembler errors involving isb/dmb instructions were resolved by specifying ARMv7-A architecture options. After EAGAIN caused missed samples in testing, the Sensor Thread was changed to retry after a short delay instead of immediately skipping the sampling cycle.

2.2. Observed Test Results

Metric	Result
Interval / duration	500 ms / 60 seconds
Samples produced / processed	117 / 117
Circular Buffer overflow	0
Alerts (T/H/S)	15 / 22 / 76
Dropped log messages	0

The application produced 117 samples against an expected 120, a deviation of 2.5%. No buffer overflow or dropped log messages were recorded in this test. The driver loaded successfully, and /proc/sms_stats displayed read counters, temperature, and humidity.



Figure 3. Project Directory Listing

3. Task 2 Memory Management and Driver Integration

3.1. Valgrind on x86_64

Criterion	Measured result
Duration	35 seconds (--mock --other)
Heap usage	21 allocs, 21 frees
Memory at exit	0 bytes in 0 blocks
Memcheck errors	0 errors

Valgrind reported that all heap blocks were freed and no leaks were possible. The SIGINT test also printed SUMMARY and returned to the shell. These conclusions apply only to the observed test runs.

```
[ALERT][1792154364002] TYPE: TEMP_SPIKE VALUE: 19.50 THRESHOLD: 5.00
[ALERT][1792154364502] TYPE: TEMP_HIGH VALUE: 43.80 THRESHOLD: 40.00
[ALERT][1792154365002] TYPE: TEMP_SPIKE VALUE: 7.80 THRESHOLD: 5.00
[ALERT][1792154365502] TYPE: TEMP_SPIKE VALUE: 10.20 THRESHOLD: 5.00
[ALERT][1792154366002] TYPE: TEMP_SPIKE VALUE: 11.50 THRESHOLD: 5.00
[ALERT][1792154367002] TYPE: TEMP_SPIKE VALUE: 11.00 THRESHOLD: 5.00
[ALERT][1792154368502] TYPE: TEMP_SPIKE VALUE: 8.30 THRESHOLD: 5.00
[ALERT][1792154369002] TYPE: HUMID_HIGH VALUE: 90.00 THRESHOLD: 80.00
[ALERT][1792154369002] TYPE: TEMP_SPIKE VALUE: 7.90 THRESHOLD: 5.00
[ALERT][1792154369502] TYPE: TEMP_HIGH VALUE: 44.00 THRESHOLD: 40.00
[ALERT][1792154369502] TYPE: TEMP_SPIKE VALUE: 20.30 THRESHOLD: 5.00
[ALERT][1792154370002] TYPE: TEMP_SPIKE VALUE: 28.00 THRESHOLD: 5.00
[ALERT][1792154370502] TYPE: TEMP_SPIKE VALUE: 6.10 THRESHOLD: 5.00
[ALERT][1792154371502] TYPE: TEMP_SPIKE VALUE: 8.50 THRESHOLD: 5.00
[ALERT][1792154372502] TYPE: TEMP_SPIKE VALUE: 5.60 THRESHOLD: 5.00
SUMMARY produced=69 consumed=69 overflow=0 alerts(T/H/S)=12/10/48 log_dropped=0
JITTER n=69 min=110 avg=254.70 max=956 p99=956 us
HIST [<100us]=0 [100-500us]=67 [500-1000us]=2 [>=1ms]=0
quyen@quyen-ThinkPad-T14-Gen-1:~/assignment02/app$ cat ~/assignment02/logs/valgrind_sms.tx
t
==11463== Memcheck, a memory error detector
==11463== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==11463== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==11463== Command: ./sms_app_x86 --mock --other -i 500 -d 35
==11463== Parent PID: 7135
==11463==
==11463==
==11463== HEAP SUMMARY:
==11463==     in use at exit: 0 bytes in 0 blocks
==11463==   total heap usage: 21 allocs, 21 frees, 15,216 bytes allocated
==11463==
==11463== All heap blocks were freed -- no leaks are possible
==11463==
==11463== For lists of detected and suppressed errors, rerun with: -s
==11463== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
quyen@quyen-ThinkPad-T14-Gen-1:~/assignment02/app$ grep -E \
'HEAP SUMMARY|in use at exit|definitely lost|indirectly lost|possibly lost|still reachable
|ERROR SUMMARY' \
~/assignment02/logs/valgrind_sms.txt
==11463== HEAP SUMMARY:
==11463==     in use at exit: 0 bytes in 0 blocks
==11463== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
quyen@quyen-ThinkPad-T14-Gen-1:~/assignment02/app$
```

Figure 4. Valgrind Results and SIGINT Test

3.2. Memory Footprint in QEMU

Time	VmRSS	VmSize
5 s	34,128 kB	37,036 kB
30 s	34,128 kB	37,036 kB
55 s	34,128 kB	37,036 kB

Both metrics remained unchanged at the three measurement points, indicating stable memory usage over the test interval. /proc/PID/maps showed mappings for text, data, heap, stack, libc.so.6, libm.so.6, and the ARM dynamic loader.

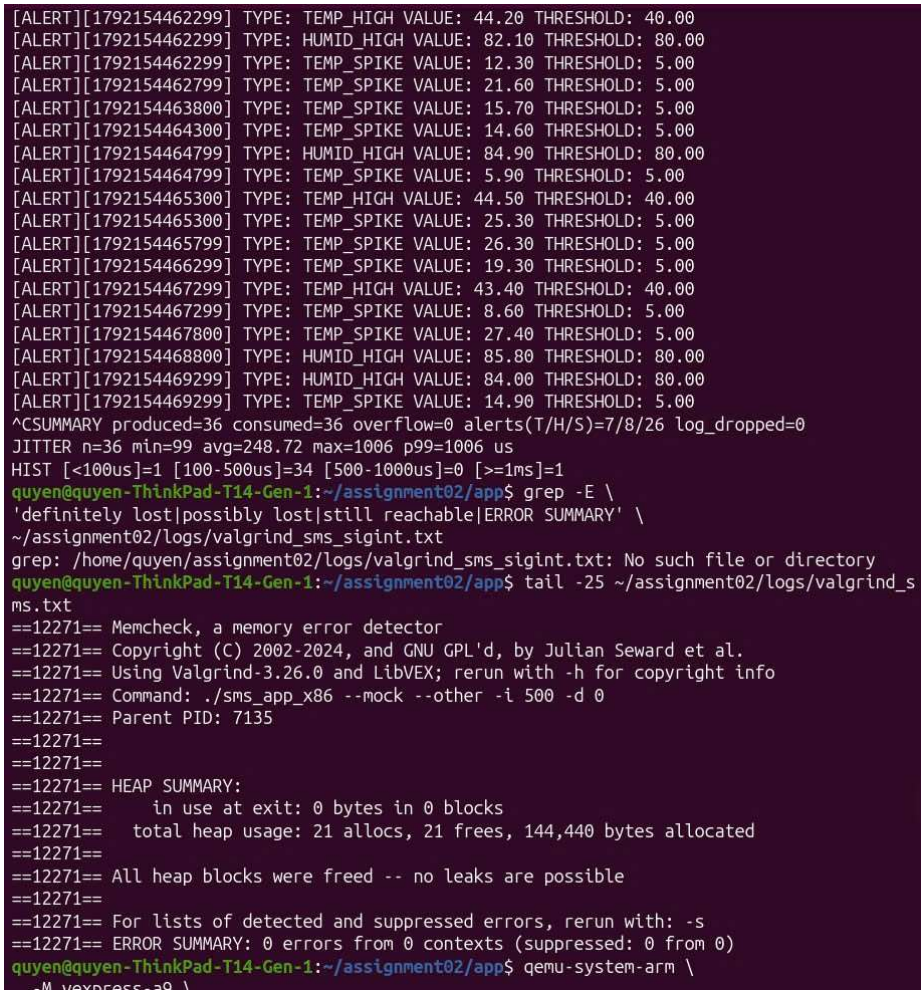


Figure 5. Additional Valgrind Output

4. Task 3 Debugging and Profiling

4.1. GDB

The x86 application was built with -O0 -g3. A breakpoint was set at ring_pop() in circular_buffer.c. GDB was used to inspect head, tail, size, produced, consumed, the thread list, and the call stack. The provided screenshots show execution stopped at ring_pop() and the thread list.

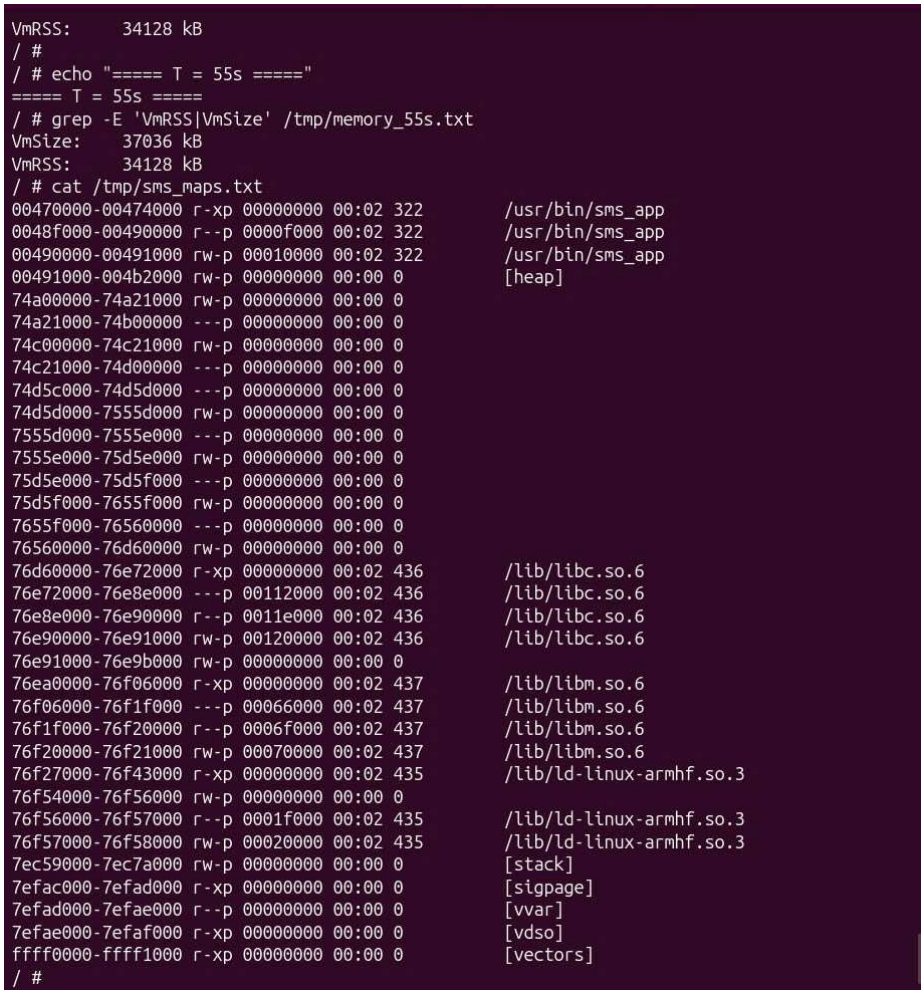


Figure 6. Process Memory Maps

4.2. strace

strace -f -c summarizes the system calls made by the application. In the provided screenshot, clock_nanosleep, futex, and mq_timedreceive account for substantial shares of reported syscall time, with 360, 428, and 315 calls respectively. These statistics describe the captured run and do not represent CPU utilization.



Figure 7. GDB Thread and Call Stack Inspection

4.3. perf

The supplied perf_sms.txt profiles sms_app_x86 on the x86 host. It contains 421 samples for the cpu/cycles/P event, an approximate event count of 45,121,660, and zero lost samples. These results do not measure execution on ARM under QEMU. The leading resolved entries are summarized below; percentages are reproduced from the top-level report rows.

Symbol	Children	Self
entry_SYSCALL_64_after_hwframe	74.25%	0.69%
alert_main	34.76%	0.98%
schedule	30.95%	0.64%
__clock_nanosleep (inlined)	26.71%	0.00%
___mq_timedreceive_time64 (inlined)	24.08%	0.00%
__vdso_clock_gettime	3.65%	3.65%

Children is the inclusive share attributed to a symbol and its callees; Self is the share attributed directly to the symbol. Inclusive rows overlap and must not be added. The syscall entry has the largest reported

Children share (74.25%), while __vdso_clock_gettime has the largest reported Self share (3.65%). alert_main has 34.76% Children but only 0.98% Self, so this does not establish that alert processing itself is computationally expensive. Sleep, message-queue receive, and scheduling paths are prominent in the sampled CPU-cycle profile; their percentages are not blocked wall-clock time or overall CPU utilization.

Limitations: 421 samples provide a small initial profile. Numerous [unknown] entries and unusual call-chain addresses limit stack attribution confidence. The log does not establish a unique algorithmic bottleneck, total runtime, IPC, cache-miss rate, or CPU utilization. Re-record a longer representative workload with suitable symbols and call-stack settings before making optimization claims.

5. Task 4 Real Time Scheduling and Jitter

The Sensor Thread was verified to use SCHED_FIFO policy=1 with real-time priority 50 through /proc/PID/task/TID/stat. A separate SCHED_OTHER test used the same 500 ms interval. During the memory-locking check, VmLck was 37,088 kB and VmRSS was 34,512 kB, providing evidence that mlockall locked memory pages.

```
[ALERT][1792155550829] TYPE: TEMP_SPIKE VALUE: 11.50 THRESHOLD: 5.00
[ALERT][1792155551829] TYPE: TEMP_SPIKE VALUE: 11.00 THRESHOLD: 5.00
SUMMARY produced=60 consumed=60 overflow=0 alerts(T/H/S)=11/9/41 log_dropped=0
JITTER n=60 min=163 avg=336.28 max=630 p99=630 us
HIST [<100us]=0 [100-500us]=54 [500-1000us]=6 [>=1ms]=0
quyen@quyen-ThinkPad-T14-Gen-1:~/assignment02/app$ cat ~/assignment02/logs/strace_sms.txt
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 40.38    0.172267         478       360           clock_nanosleep
 39.38    0.168016         392       428        17 futex
 16.89    0.072075         228       315       254 mq_timedreceive
  1.11    0.004752          66        72           write
  0.55    0.002345          18       124           lseek
  0.46    0.001945          31        61           mq_timedsend
  0.25    0.001066        1066         1           execve
  0.19    0.000826          59        14           mmap
  0.14    0.000613         153         4           clone3
  0.11    0.000466          27        17           rt_sigprocmask
  0.08    0.000335          67         5           munmap
  0.07    0.000294          36         8           madvise
  0.07    0.000290          58         5           rseq
  0.06    0.000246          61         4           openat
  0.05    0.000225          45         5           set_robust_list
  0.03    0.000145          29         5           mprotect
  0.03    0.000121          24         5           close
  0.02    0.000094          15         6           fstat
  0.02    0.000089          44         2           mq_unlink
  0.02    0.000065          21         3           brk
  0.01    0.000059          19         3           rt_sigaction
  0.01    0.000058          58         1           mq_open
  0.01    0.000047          15         3           read
  0.01    0.000027          13         2           pread64
  0.01    0.000027          27         1         1 mlockall
  0.01    0.000024          24         1           munlockall
  0.00    0.000021          21         1           getrandom
  0.00    0.000019          19         1         1 access
  0.00    0.000016          16         1           mq_getsetattr
  0.00    0.000014          14         1           set_tid_address
  0.00    0.000013          13         1           arch_prctl
  0.00    0.000012          12         1           prlimit64
------ ----------- ----------- --------- --------- ----------------
100.00    0.426612         292      1461       273 total
quyen@quyen-ThinkPad-T14-Gen-1:~/assignment02/app$
```

Figure 8. System Call Summary from strace

Metric	SCHED_OTHER	SCHED_FIFO
Jitter samples	1,039	1,040
Min (µs)	427	360
Average (µs)	5,032.42	5,054.56
Max (µs)	9,811	9,997
P99 (µs)	9,554	9,647
Histogram <100 / 100–500 / 500–1000 / ≥1000 µs	0 / 3 / 64 / 972	0 / 6 / 50 / 984

FIFO P99 was 93 µs higher than OTHER, approximately 0.97% in this experiment. Because execution takes place in QEMU, wake-up timing is also affected by host CPU scheduling. This result does not establish that FIFO always performs worse than OTHER.

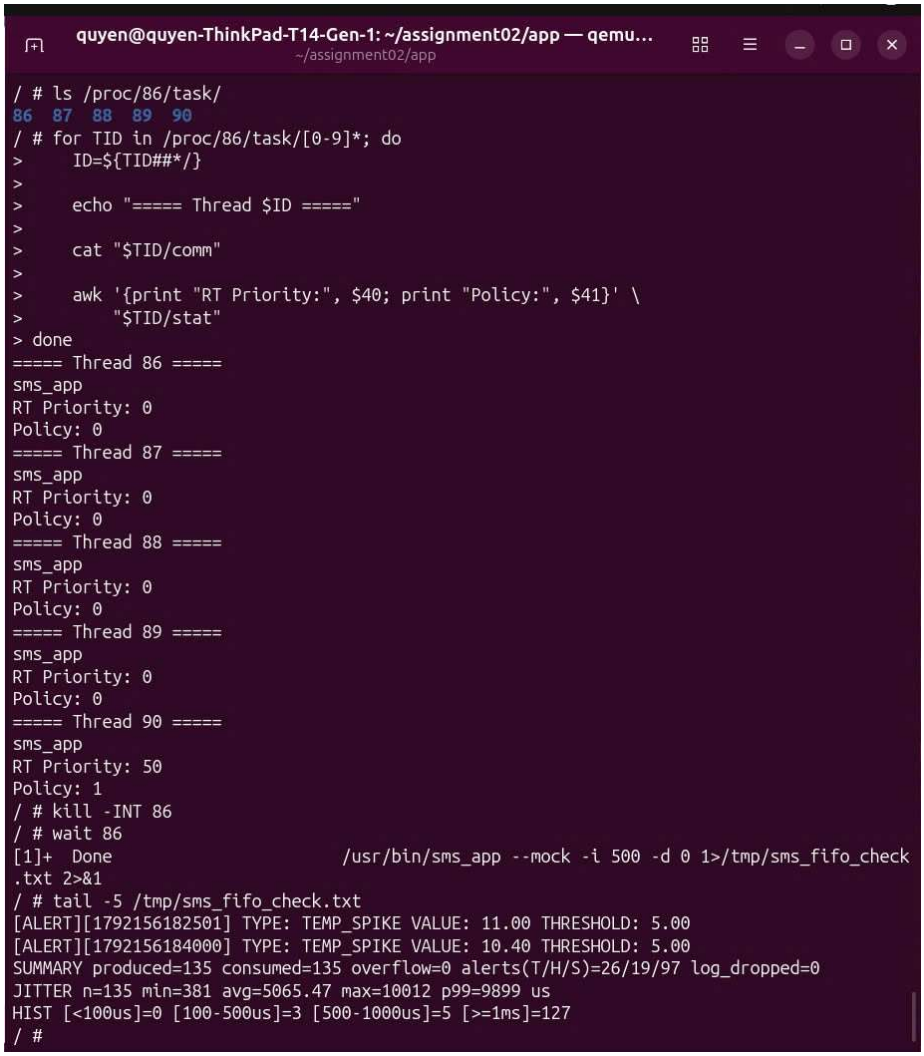


Figure 9. Two QEMU Jitter Measurements with More Than 1000 Samples Each

6. Task 5 Conclusion

The system successfully integrates a kernel module, four threads, a Circular Buffer, a POSIX Message Queue, a Logger, and a shutdown mechanism. Testing showed no Valgrind errors, stable memory usage

over the observed interval, and more than 1,000 jitter samples for each scheduling policy. The supplied host perf log also provides an initial CPU-cycle profile.

Suggested Improvements

Repeat jitter measurements to assess run-to-run variation, test PREEMPT_RT, improve EAGAIN handling, and perform more extensive thread synchronization tests. For profiling, evaluate the proposed reduction in unnecessary wake-ups and repeated output, then compare equivalent workloads before and after the change.